

ArchitectureScope

Platform Guide

What the project is, how it is built, how to run and test it, how the API docs work, and how it will be deployed and scaled.



Version	0.4 (platform rebuild — Phase 1 complete)
Branch	rebuild/platform
Date	October 2026
Written for	a developer on their first project: every step is spelled out

Read-only against HubSpot by design. Nothing in this project creates, updates or deletes portal data.

Contents

1. Start here	5
1.1 How to read this guide	5
1.2 Conventions used in this guide	5
1.3 What you need installed	5
2. What ArchitectureScope is	7
2.1 The modules	7
2.2 The read-only promise	7
2.3 Where AI is used	8
3. Where the project is today	9
3.1 The migration strategy: "strangler"	9
3.2 What Phase 1 delivered	9
3.3 Roadmap	9
4. Vocabulary you will need	11
5. Architecture	12
5.1 The big picture today	12
5.2 The target platform	13
5.3 The layers and the rules between them	13
5.4 Ports and adapters (why the domain never sees HubSpot)	14
5.5 Tour of the folders	14
5.6 Life of a request, step by step	15
5.7 How errors look	15
5.8 Editable configuration files	16
6. Set up your computer (one time)	17
7. Run the program	19
7.1 Run the Streamlit app	19
7.2 Run the API	19
8. Using the API documentation (Swagger)	21
8.1 Open the docs tab	21
8.2 Your first request: the health check	21
8.3 Generate a blueprint (a POST with a body)	22
8.4 Draw the architecture diagram	22
8.5 See a validation error on purpose	22
8.6 Proposals: start, edit, analyse	23

8.7 Discovery endpoints	23
8.8 Endpoints that read a live HubSpot portal	23
8.9 Calling the API without Swagger	24
8.10 The contract file: openapi.json	24
9. Testing	25
9.1 The kinds of tests in this project	25
9.2 Run the tests	25
9.3 Worked example: change a Discovery dropdown and test it	26
9.4 Golden tests: when an output change is intended	26
9.5 The other quality checks (what CI runs)	26
10. Making changes safely (Git and CI)	28
10.1 The everyday Git workflow	28
10.2 What CI runs	29
10.3 Adding a new API endpoint (the recipe)	29
11. Configuration and secrets	30
12. How it will be deployed	31
12.1 Today	31
12.2 The target: containers	31
12.3 Environments	31
12.4 The delivery pipeline	32
12.5 Hosting options	32
12.6 Database changes: migrations	32
12.7 Production checklist (before the first real launch)	33
13. Scaling and escalation	34
13.1 Why it can scale	34
13.2 The scaling path, step by step	34
13.3 Observability: seeing what production is doing	35
13.4 Escalating incidents (when something breaks)	35
14. Troubleshooting	36
15. Cheat sheet	37
Setup (once)	37
Run	37
Check everything before you push	37
Git	37
Useful addresses	37

Appendix A — Endpoint reference 38

Appendix B — Key files 40

1. Start here

This guide explains ArchitectureScope from zero: what it does, how the code is organized, how to run it on your computer, how to use the API documentation page, how to test changes, and how the project will be deployed and scaled. It assumes this is your first software project, so it defines the vocabulary and spells out every click and command.

1.1 How to read this guide

- **Sections 2–5** explain *what* the project is and *why* it is built the way it is. Read them once, slowly.
- **Sections 6–9** are hands-on: setup, running the app, the API docs, and testing. Keep the guide open next to VS Code and do each step as you read it.
- **Sections 10–13** cover changing the code safely, configuration, deployment and scaling — read them before your first real change.
- **Section 14** (Troubleshooting) and **Section 15** (Cheat sheet) are for later, when something goes wrong or you just need a command.

1.2 Conventions used in this guide

Commands you type are shown in grey boxes like this one:

```
python --version
```

To run a command: click inside the terminal, type (or paste) the text **exactly**, then press **Enter**. Wait until the terminal shows the prompt again (a line ending in `>`) before typing the next command. Some commands print a lot of text; that is normal.

TIP To paste into the VS Code terminal use **Ctrl+V** (or right-click → Paste). To copy text *from* the terminal, select it with the mouse and press **Ctrl+C** — but be careful: if nothing is selected, **Ctrl+C stops the running program**.

Words in `this font` are file names, folder names, commands or values you type. Coloured boxes mark tips (green), warnings (orange) and background knowledge (purple).

1.3 What you need installed

Tool	What it is for	How to check it
VS Code	The editor you use to open the project, read code and run the terminal.	You already have it — the project is open in it.
Python 3.12 or newer	The language the backend is written in.	<code>python --version</code> → should print 3.12, 3.13 or 3.14.
Git	Version control: saves the history of every change.	<code>git --version</code>
uv	Installs the project's Python libraries in an isolated folder (<code>.venv</code>), exactly the versions in <code>uv.lock</code> .	<code>python -m uv --version</code> (Section 6 installs it).
A web browser	To see the app and the API docs.	Chrome, Edge or Firefox.

Tool	What it is for	How to check it
<i>Docker Desktop (later)</i>	Runs the database and Redis on your machine. Needed from Phase 2.	Not needed yet.

2. What ArchitectureScope is

ArchitectureScope is a toolkit for **HubSpot solutions architects** — the people who design and implement HubSpot (a CRM: customer-relationship-management software) for client companies. It takes a project from the first discovery call with a client to the final hand-off: it captures requirements, audits the client's existing HubSpot portal, designs the target architecture, drafts the proposal and Statement of Work, and produces the documentation and test plans.

2.1 The modules

#	Module	What it does	Talks to
11	Discovery Call Assistant	Guided form used live during the client call (dropdowns, notes, follow-up flags). AI turns the notes into a business-analysis document and a plain-language explainer.	Anthropic (AI)
6	Requirements Document	The Written Requirements Document (WRD): goals, numbered requirements, risks, open questions.	—
7	Joint Evaluation Plan	Who is involved on both sides, technical/business needs, milestones.	—
2	Portal Auditor	Checks a live portal against best-practice rules (duplicates, naming, risky workflows, orphaned records).	HubSpot (read)
2B	Property Audit	Rates every property Keep / Review / Remove candidate, with an Excel and PDF report.	HubSpot (read)
3	Documentation Generator	Pulls a snapshot of the portal (objects, properties, pipelines, workflows, users) as a document.	HubSpot (read)
4	Executive Report	Plain-language health report for the client, plus a detailed internal version.	—
1	Architecture Generator	Generates the target architecture: custom objects, properties, associations, pipelines, workflows.	—
5	Architecture Diagram	Draws the architecture (from a blueprint or a live portal) as PNG, Graphviz or draw.io.	—
10	Proposal & SOW Builder	Builds the branded proposal / Statement of Work PDF with scope, pricing and coverage checks.	—
8	Test Case Document	Test plan derived from the requirements.	—
9	Project Library	Archive of everything generated, per client.	local files

2.2 The read-only promise

ArchitectureScope connects to *real client portals*. A tool that accidentally changed or deleted a client's data would be a disaster, so the project is **read-only against HubSpot by design** — and this is enforced in code, not just promised:

- The HubSpot client only has `get_...` methods. There is no generic "send any request" method it could misuse.

- An automated test (`packages/integrations/tests/test_hubspot.py`) fails if anyone ever adds a method that isn't a read.
- The domain code can only reach HubSpot through an interface (`PortalReader`) that has no write methods at all.

2.3 Where AI is used

Only the Discovery Call Assistant uses AI (Anthropic's Claude). It turns the specialist's raw call notes into a structured document. The AI is never asked to invent facts: blank answers become follow-up questions. Everything else in the project is deterministic code — the same input always gives the same output, which is what makes it testable.

3. Where the project is today

ArchitectureScope started as a **Streamlit** app. Streamlit is a Python library that turns a script into a web page very quickly — perfect for a prototype, but limited for a production product: every click re-runs the whole page, work is lost when the browser tab refreshes, and the design options are narrow.

The project is now being rebuilt into a **professional platform**: a modern web interface (Next.js), a separate backend API (FastAPI), a database (PostgreSQL), automated tests and automatic deployment. This guide describes that platform.

3.1 The migration strategy: "strangler"

We are not throwing the old app away and starting over (a *rewrite*), because that is risky: for months nothing works and bugs from the old app silently get lost. Instead we use the **strangler** pattern — named after a vine that grows around a tree until it replaces it:

1. Pull the business logic out of the Streamlit app into a shared package (**done** — Phase 1).
2. Build the new API and web app on top of that same package, one module at a time.
3. While that happens, the old Streamlit app keeps working, using the *same* business logic — so both always give identical results.
4. When a module exists in the new web app, its Streamlit page is retired. When all are retired, Streamlit is deleted.

3.2 What Phase 1 delivered

- **A golden baseline.** Before changing anything, the outputs of every module were recorded ("golden files"). Tests compare against them forever, proving nothing changed by accident.
- **The domain package** (`packages/domain`): all business rules, with no web framework inside.
- **The integrations package** (`packages/integrations`): the read-only HubSpot client.
- **The FastAPI service** (`apps/api`): 31 API endpoints with interactive documentation.
- **Automated quality checks:** 277 tests, type checking, linting, architecture rules, and a GitHub Actions workflow that runs them all on every change.

GOOD TO KNOW Phase 1 changed nothing you can see in the Streamlit app — that was the goal. The new look arrives in Phase 3 with the Next.js web app.

3.3 Roadmap

Phase	What	Status
1	Domain extraction, FastAPI service, typed API contract, golden tests, CI	Done
2	PostgreSQL database, Alembic migrations, Redis, file storage, Google sign-in with roles, every table carries <code>org_id</code>	Next (needs Docker Desktop)
3	Next.js web app: design system, project workspace, command palette, dark mode; Discovery as the first module (with Discovery Phases 2–4: rules, required-tasks checklist, build order)	Planned

Phase	What	Status
4	Remaining modules in the web app, dashboards, interactive diagrams, Playwright end-to-end tests, Docker images, deployment	Planned
5	Retire the Streamlit app	Planned

4. Vocabulary you will need

Keep this page handy. Every later section uses these words.

Term	Meaning, in plain words
Frontend	The part users see and click: web pages in the browser. Today: Streamlit. Future: Next.js.
Backend	Code running on a server that does the real work and returns data. Here: the FastAPI service + domain package.
API	<i>Application Programming Interface</i> — a set of URLs a program can call to ask the backend to do something. Think of it as a restaurant menu for programs.
Endpoint	One item on that menu: a method + a URL, e.g. <code>POST /api/v1/blueprints</code> .
HTTP method	The verb of a request. <code>GET</code> = read something; <code>POST</code> = send data to be processed.
JSON	The text format the API speaks: <code>{"project_name": "Acme", "regions": ["US"]}</code> .
Status code	A number in every response: <code>200</code> OK · <code>400</code> you sent something wrong · <code>422</code> invalid fields · <code>404</code> not found · <code>500</code> server bug · <code>502</code> / <code>503</code> an outside service failed or isn't configured.
Schema	The exact shape of a JSON message: which fields, which types, which are required.
Domain / business logic	The rules of the business (how an audit rates a property, how pricing adds up), independent of any website or database.
Package	A folder of Python code that can be installed and imported by other code.
Monorepo	One Git repository holding several apps and packages that work together.
Virtual environment (.venv)	A private folder of Python libraries just for this project, so projects don't interfere with each other.
Dependency / lockfile	A library the project uses. <code>uv.lock</code> pins the exact version of every one, so everyone (and CI) installs the same thing.
Test	A small program that checks the real program gives the expected result.
Golden test	A test comparing today's output with a saved, approved output (the "golden file").
CI (Continuous Integration)	A robot (GitHub Actions) that runs all checks on every push and reports green or red.
Docker / container	A way to package an app with everything it needs so it runs the same on any computer or server.
Database (PostgreSQL)	Where data is stored permanently (projects, users, documents).
Redis	A very fast in-memory store, used for caching, sessions and job queues.
SSE (Server-Sent Events)	A way for the server to stream text to the browser piece by piece — used so AI drafts appear as they are written.
CORS	A browser security rule: which websites are allowed to call the API.
Environment variable / .env	Settings (like API keys) given to a program from outside its code. <code>.env</code> is a local file that holds them; it is never committed.

5. Architecture

5.1 The big picture today

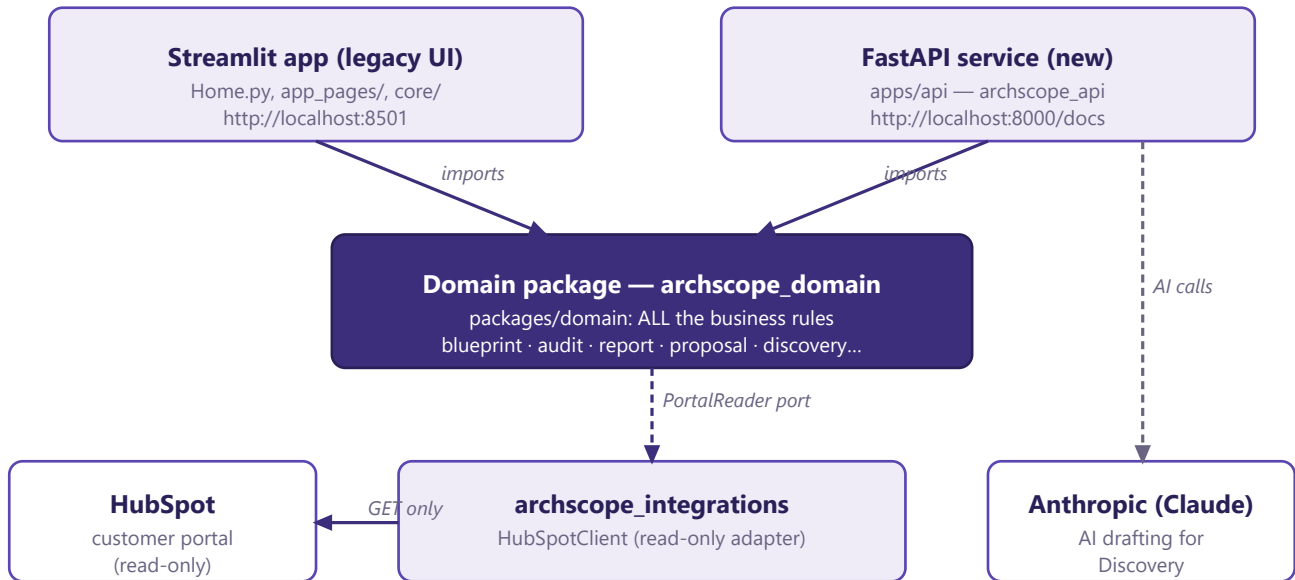


Figure 1 — After Phase 1: two front doors (Streamlit and the API) share one domain package.

The most important idea in the whole project: **business rules live in exactly one place** — the domain package. The Streamlit app and the API are just two different *doors* into it. That is why they can never disagree, and why the web app in Phase 3 will be safe to build.

5.2 The target platform

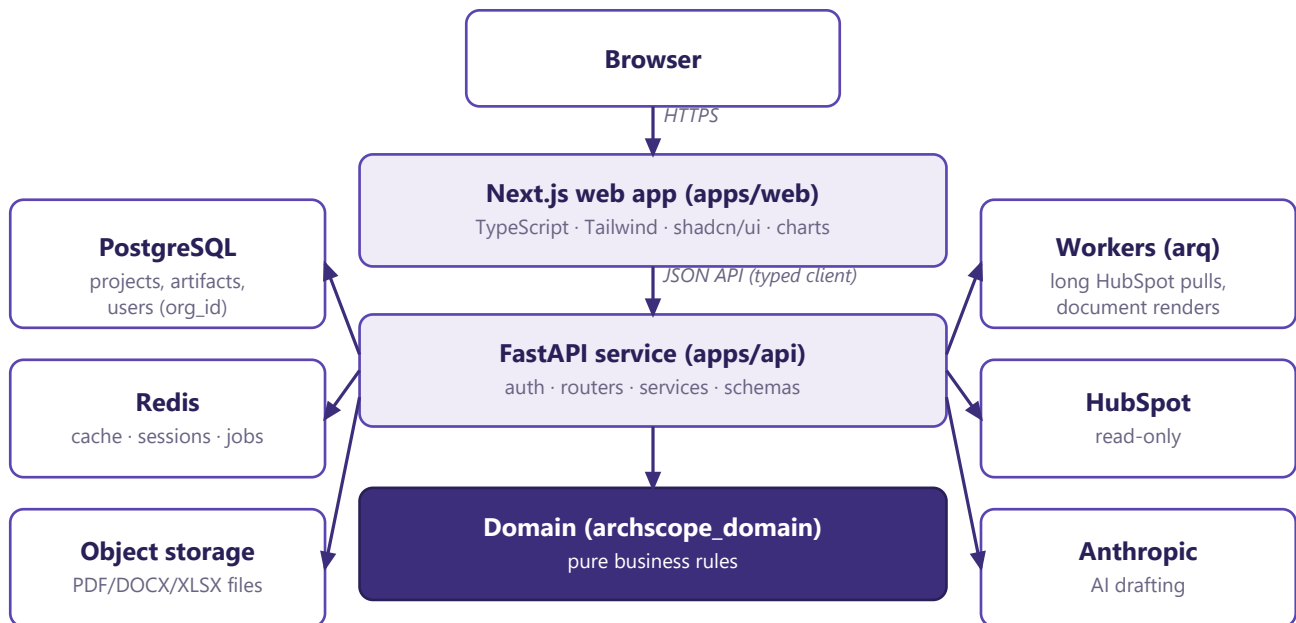


Figure 2 — Target architecture after Phases 2–4.

- The **browser** loads the Next.js web app.
- The **web app** calls the API using a TypeScript client *generated* from the API's contract, so a typo in a field name is caught before the code even runs.
- The **API** checks who you are (Google sign-in), loads/saves data in **PostgreSQL**, caches in **Redis**, stores generated files in **object storage**, and hands slow work (big portal pulls) to **workers**.
- All decisions are still made by the **domain** package at the bottom.

5.3 The layers and the rules between them

Large projects stay maintainable when each part has one job and dependencies only point *inward*, toward the business rules. This project enforces that automatically with a tool called **import-linter** (configured in `pyproject.toml`). If someone breaks a rule, CI turns red.

Layer	Folder	Its one job	May NOT import
Domain	<code>packages/domain</code>	Business rules: audits, pricing, blueprints, reports, discovery.	Streamlit, FastAPI, databases, HTTP clients, HubSpot SDK, Anthropic SDK
Integrations	<code>packages/integrations</code>	Talk to HubSpot (read-only), implementing the domain's <code>PortalReader</code> port.	The API, Streamlit, FastAPI
API · routers	<code>apps/api/.../routers</code>	HTTP: read the request, call a service, choose the status code.	—
API · services	<code>apps/api/.../services</code>	Use cases: orchestrate domain functions and adapters.	HTTP schemas
API · schemas	<code>apps/api/.../schemas</code>	The public contract: JSON shapes, plus mapping to/from domain objects.	Services

Layer	Folder	Its one job	May NOT import
Legacy UI	Home.py , app_pages/ , core/	Streamlit pages (being retired).	—

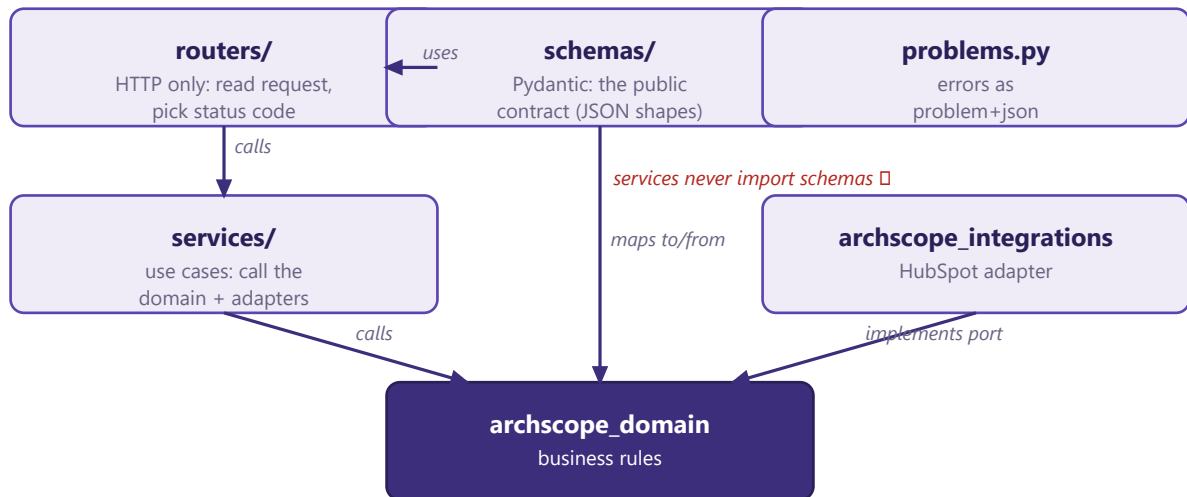


Figure 3 — Inside the API. Arrows show allowed calls.

5.4 Ports and adapters (why the domain never sees HubSpot)

The domain needs portal data, but must not depend on HubSpot's library. So the domain defines a **port** — an interface listing what it needs (`get_properties` , `get_records_sample` , ...) in `packages/domain/src/archscope_domain/ports.py` . The HubSpot client is one **adapter** that fulfils it. Tests use a fake in-memory adapter instead. Benefits: the domain is testable without internet, and if HubSpot ever changes its library only the adapter changes.

5.5 Tour of the folders

```

architecturescope/
├── packages/
│   ├── domain/
│   │   ├── src/archscope_domain/
│   │   │   ├── config/
│   │   │   └── tests/
│   │   └── integrations/
│   ├── apps/
│   │   └── api/
│   │       ├── src/archscope_api/
│   │       ├── openapi.json
│   │       └── tests/
│   ├── Home.py, app_pages/, core/
│   ├── tests/
│   ├── sample_inputs/
│   ├── pyproject.toml, uv.lock
│   └── .github/workflows/
└── business rules (archscope_domain)
    ├── blueprint.py, audit.py, report.py, proposal.py, ...
    ├── rules.yaml, discovery_options.yaml <- editable settings
    ├── unit tests + golden/ (approved outputs)
    └── HubSpotClient (read-only)

FastAPI service (archscope_api)
main.py, routers/, services/, schemas/, problems.py
the API contract (generated, committed)

legacy Streamlit app
Streamlit smoke tests
example_project.json (great for trying the API)
workspace settings + exact library versions
CI definition (ci-python.yml)

```

5.6 Life of a request, step by step

What happens when someone calls `POST /api/v1/blueprints` with a project description:

1. **Middleware** gives the request an ID (header `X-Request-ID`) so it can be traced in logs and error messages.
2. **FastAPI + the schema** (`schemas/blueprint.py`, class `BlueprintInput`) validate the JSON: right fields, right types, no unknown fields. If anything is wrong, the request stops here with `422`.
3. **The router** (`routers/v1/architecture.py`) converts the validated input into a domain object and calls the service.
4. **The service** (`services/architecture.py`) runs the domain's `BlueprintGenerator` with the rules from `rules.yaml`. CPU-heavy work runs in a background thread so the server keeps answering other requests.
5. **The router** converts the domain result back into the response schema (`Blueprint`) and FastAPI returns it as JSON with status `200`.

5.7 How errors look

Every error from the API has the same shape — the internet standard **RFC 9457 "problem details"** (content type `application/problem+json`). The `type` field is stable, so programs can react to it; `trace_id` matches the request ID.

```
{
  "type": "https://architecturescope.dev/problems/validation-error",
  "title": "Request validation failed",
  "status": 422,
  "detail": "1 invalid field(s).",
  "instance": "/api/v1/blueprints",
  "trace_id": "4f1c...",
  "errors": [{"loc": ["body", "project_name"], "msg": "String should have at least 1 character"}]
}
```

type ends with	Status	Meaning
<code>validation-error</code>	422	The JSON you sent is missing fields or has wrong types. Read <code>errors</code> .
<code>hubspot-token-required</code>	400	This endpoint reads a portal; add the token (Section 8.8).
<code>hubspot-access-denied</code>	502	HubSpot rejected the token (wrong/expired token or missing scope).
<code>hubspot-unavailable</code>	502	HubSpot failed or rate-limited us repeatedly.
<code>ai-not-configured</code>	503	No <code>ANTHROPIC_API_KEY</code> on the server (Section 11).
<code>not-found</code>	404	The URL doesn't exist — check for typos.
<code>internal-error</code>	500	A bug. Details are hidden from callers on purpose; look at the server log with the <code>trace_id</code> .

5.8 Editable configuration files

Two YAML files hold rules that non-programmers can change without touching code:

- `packages/domain/src/archscope_domain/config/rules.yaml` — best-practice rules used by the Architecture Generator and the Portal Auditor (naming conventions, pipeline templates, thresholds).
- `packages/domain/src/archscope_domain/config/discovery_options.yaml` — every Discovery question, its type (dropdown, multi-select, radio, text) and its options. Section 9.3 walks through editing it.

6. Set up your computer (one time)

Do this once. Afterwards, Section 7 is all you need each day.

1 Open the project in VS Code

Start VS Code. Choose **File** → **Open Folder...**, select the `architecturescope` folder (in `Escritorio`), and click **Select Folder**.

2 Open a terminal inside VS Code

From the top menu choose **Terminal** → **New Terminal** (or press **Ctrl+`** — the backtick key, left of the 1 key). A panel opens at the bottom. Its prompt ends with the project path, for example `PS C:\Users\aylin\OneDrive\Escritorio\architecturescope>`. Every command in this guide is typed here.

WATCH OUT If the prompt shows a *different* folder, move to the project first with `cd "$HOME\OneDrive\Escritorio\architecturescope"` and press Enter. Commands run from the wrong folder fail with "file not found".

3 Check Python

Type the command below and press Enter. It must print 3.12 or higher.

```
python --version
```

If you see "Python was not found", install Python from python.org (tick **Add python.exe to PATH** in the installer), then close and reopen VS Code.

4 Install uv (the package installer)

uv installs the exact library versions this project needs into a private `.venv` folder.

```
python -m pip install --user uv
python -m uv --version
```

The second command should print something like `uv 0.12.x`. In this guide we always type `python -m uv` — it works even if Windows doesn't know where uv was installed.

5 Make sure you are on the right branch

A *branch* is a parallel line of work in Git. The platform work lives on `rebuild/platform`.

```
git status
git switch rebuild/platform
```

`git status` prints `On branch rebuild/platform` when you are in the right place.

6 Install the project's libraries

This reads `uv.lock` and installs everything (FastAPI, Streamlit, pytest, ...) into `.venv`. The first run takes a minute or two.

```
python -m uv sync --all-packages
```

7 Create your local settings file (.env)

In the VS Code file explorer (left side), check whether a file named `.env` exists in the project root. If not, create it (**right-click the empty area** → **New File**, name it `.env`) with this content:

```
AUTH_DISABLED=1
# Optional – only needed for the AI drafting in Discovery:
# ANTHROPIC_API_KEY=sk-ant-...
```

WATCH OUT `AUTH_DISABLED=1` skips the sign-in screen **on your computer only**. Never use it on a real server. The `.env` file is listed in `.gitignore`, so Git will never upload it — keep it that way, because it can hold secret keys.

8 Prove everything works

Run the full test suite (Section 9 explains it in detail).

```
python -m uv run pytest -q
```

After about 30 seconds the last line should read `277 passed`. If it does, your setup is complete.

7. Run the program

There are two programs. You can run either one, or both at the same time in two terminals.

Program	What you see	Address
Streamlit app	The current user interface (all 11 modules).	http://localhost:8501
FastAPI service	The new backend and its interactive documentation.	http://localhost:8000/docs

GOOD TO KNOW `localhost` means "this computer". The number after the colon is the **port** — like an apartment number in a building. Each program listens on its own port.

7.1 Run the Streamlit app

1 Open a terminal

Terminal → **New Terminal** in VS Code (check you are in the project folder).

2 Start Streamlit

```
python -m uv run streamlit run Home.py
```

Wait for the line `Local URL: http://localhost:8501`. Your browser usually opens by itself; if not, open the browser and type `http://localhost:8501` in the address bar.

3 Use it

Start from **Setup & API keys** in the sidebar. The Discovery Call Assistant works without any key for note-taking; HubSpot pages need a private app token.

4 Stop it

Click in the terminal running Streamlit and press **Ctrl+C**. The prompt comes back.

7.2 Run the API

1 Open a second terminal

Click the **+** icon at the top right of the terminal panel. You now have two terminals (listed on the right). Leave Streamlit running in the first one if you want both.

2 Start the API

```
python -m uv run uvicorn archscope_api.main:app --reload
```

Wait for `Uvicorn running on http://127.0.0.1:8000` . The `--reload` option restarts the server automatically whenever you save a code file — handy while developing.

3 Check it is alive

Open the browser and visit `http://localhost:8000/healthz` . You should see:

```
{"status":"ok","version":"0.4.0","environment":"local"}
```

4 Stop it

In that terminal press **Ctrl+C**.

TIP If the browser says "This site can't be reached", the server isn't running (or has stopped) — look at its terminal for an error, or start it again.

8. Using the API documentation (Swagger)

FastAPI automatically builds an **interactive documentation page** from the code — called *Swagger UI*. It lists every endpoint, shows exactly what JSON each one expects and returns, and lets you **send real requests with a button**, no programming needed. It is the best way to explore and test the API.

8.1 Open the docs tab

1 Start the API

Follow Section 7.2 so that `Uvicorn running on http://127.0.0.1:8000` is showing.

2 Open the docs

In your browser's address bar type `http://localhost:8000/docs` and press Enter.

3 Get oriented

You will see:

- **At the top:** the title *ArchitectureScope API*, the version, and a link to `/openapi.json` (the machine-readable contract).
- **An Authorize button** (padlock icon, top right): used to give a HubSpot token to endpoints that read a portal.
- **Groups of endpoints** (tags): *health, architecture, portal, proposals, discovery, exports*. Each row shows a coloured method (blue `GET`, green `POST`), the URL, and a one-line summary.
- **Schemas** (at the very bottom): every JSON shape the API uses. Click one to expand it and see its fields, types and descriptions.

8.2 Your first request: the health check

1 Expand the endpoint

Click the row **GET /healthz**. It expands to show parameters (none) and possible responses.

2 Click "Try it out"

The button is on the right of the expanded panel.

3 Click "Execute"

The big blue button. Swagger sends the request to your running server.

4 Read the result

Below, under **Responses**, you will find:

- **Curl** — the same request written as a terminal command (useful to share or automate).
- **Request URL** — the exact address that was called.

- **Server response** → **Code:** `200` ; **Response body:** the JSON; **Response headers:** including `x-request-id` .

8.3 Generate a blueprint (a POST with a body)

1 Copy the sample project

In VS Code open `sample_inputs/example_project.json` . Press **Ctrl+A** (select all) then **Ctrl+C** (copy).

2 Open the endpoint

In Swagger, under **architecture**, click **POST /api/v1/blueprints**, then **Try it out**.

3 Paste the body

Click inside the **Request body** box, press **Ctrl+A** to select the example JSON Swagger put there, then **Ctrl+V** to replace it with the sample project.

4 Execute and read

Click **Execute**. Code `200` and a large JSON response: the generated custom objects, properties, associations, pipelines and suggested workflows (each with `overwrite_risk`).

TIP Scroll inside the response box, or click the small **Download** button next to it to save the JSON as a file.

8.4 Draw the architecture diagram

1. Copy the **entire response body** from Section 8.3 (click in the response box, Ctrl+A, Ctrl+C).
2. Open **POST /api/v1/diagrams/from-blueprint** → **Try it out**. Replace the body with `{"blueprint":` + paste + `}` — i.e. wrap the copied JSON inside `{"blueprint": ... }` . Execute.
3. Copy that response (it has `nodes` , `edges` , `dangling_edges`).
4. Open **POST /api/v1/diagrams/render** → **Try it out**. Set the **format** dropdown to `png` . Body: `{"diagram":` + paste + `, "show_labels": true}` . Execute.
5. Swagger shows a **Download file** link. Click it to save `architecture_diagram.png` and open it.

8.5 See a validation error on purpose

Open **POST /api/v1/blueprints** → **Try it out**, and send this body:

```
{"project_name": "", "unknown_field": 1}
```

You get code `422` with a problem+json body listing two errors: the name is too short, and `unknown_field` is not allowed. The API **rejects** unknown fields instead of silently ignoring them — so a typo in a field name never goes unnoticed.

8.6 Proposals: start, edit, analyse

1. Run **GET /api/v1/proposals/default** (Try it out → Execute). You receive a complete blank proposal with SonaMation boilerplate. Copy it.
2. Open **POST /api/v1/proposals/analysis** → Try it out. Body: `{"proposal":` + paste + `}`.
3. Before executing, edit a milestone fee: find `"fee": 0.0` in the first milestone and change it to `"fee": 5000`. Execute.
4. The response shows `pricing` (total 5000), `coverage`, `scope_by_status` and `warnings` such as missing acceptance criteria.
5. To get the PDF: **POST /api/v1/proposals/pdf** with the same body → **Download file**.

8.7 Discovery endpoints

- **GET /api/v1/discovery/question-bank** — all questions and dropdown options, read live from `discovery_options.yaml`.
- **POST /api/v1/discovery/business-analysis/payload** — shows exactly what would be sent to the AI, *without* calling it. Try the body `{"client_name": "Acme", "business_type": "dental"}`.
- **POST /api/v1/discovery/business-analysis** and **/client-explainer** — the real AI drafting, streamed as Server-Sent Events. They need `ANTHROPIC_API_KEY` (Section 11); without it you get `503 ai-not-configured`. Swagger displays the stream only after it finishes, as lines like `event: delta / data: {"text": ...}`; the future web app will show the text appearing live.

8.8 Endpoints that read a live HubSpot portal

`/portal/snapshot`, `/audits` and `/property-audits` need a HubSpot **private app token**:

1. Click **Authorize** (padlock, top right of the page).
2. In **HubSpotToken (apiKey)**, paste the token into **Value**, click **Authorize**, then **Close**. The padlocks on those endpoints now look closed.
3. Run **POST /api/v1/portal/snapshot** → Try it out → Execute. You receive the portal's objects, properties, pipelines, workflows, owners and teams.
4. Copy the response and use it as `{"snapshot": ...}` for `/audits`, `/property-audits`, `/reports/context` or `/diagrams/from-snapshot`.

WATCH OUT The token is only kept in your browser tab and sent with each request; the API never stores or logs it. Still: never paste real client tokens into screenshots, chats or files you commit.

8.9 Calling the API without Swagger

Programs (and the future web app) call the same URLs. From PowerShell you can try:

PowerShell (the backtick ` at the end of a line continues the command on the next line)

```
Invoke-RestMethod http://localhost:8000/healthz

$body = Get-Content sample_inputs\example_project.json -Raw
$bp = Invoke-RestMethod -Method Post -Uri http://localhost:8000/api/v1/blueprints `
      -ContentType "application/json" -Body $body
$bp.custom_objects | Select-Object name, plural_label
```

8.10 The contract file: openapi.json

Swagger is drawn from a machine-readable description of the API called **OpenAPI**, saved in `apps/api/openapi.json`. It is the *contract* between backend and frontend: in Phase 3 the web app's TypeScript code is generated from it. If you change an endpoint, regenerate it (Section 9.5); CI fails if you forget.

GOOD TO KNOW In production the docs page is switched off (`ARCHSCOPE_ENVIRONMENT=production`), so the API's map isn't public. Developers use it locally or on staging.

9. Testing

Tests are small programs that check the real program. They let you change code with confidence: if you break something, a test fails *immediately*, on your computer, instead of a client discovering it later.

9.1 The kinds of tests in this project

Kind	Where	What it proves	Count
Unit tests	<code>packages/domain/tests/</code>	Each business rule works (ratings, pricing, naming checks...).	164
Golden tests	<code>packages/domain/tests/golden/</code>	Outputs are identical to the approved baseline captured from the original app.	10 (inside the 164)
Adapter tests	<code>packages/integrations/tests/</code>	HubSpot client is read-only, retries rate limits, maps errors.	12
API tests	<code>apps/api/tests/</code>	Every endpoint over HTTP: golden parity, exact mapping, errors, streaming, contract.	52
App smoke tests	<code>tests/</code>	Every Streamlit page still opens and key flows work.	~49

9.2 Run the tests

1 Open a terminal in the project folder

(Section 6, steps 1–2.) You don't need the API or Streamlit running — tests start what they need.

2 Run everything

```
python -m uv run pytest
```

3 Read the output

Each dot `.` is a passing test. `F` = a test failed (wrong result), `E` = error (the test itself crashed), `s` = skipped. The last line summarizes, e.g. `277 passed in 28s`. Anything failed? pytest prints the test's name and a diff of *expected* vs. *actual* above the summary.

Useful variations

```
python -m uv run pytest -q           # quieter output
python -m uv run pytest packages     # only domain + integrations
python -m uv run pytest apps/api/tests # only the API
python -m uv run pytest packages/domain/tests/golden # only golden parity
python -m uv run pytest -k proposal  # only tests with "proposal" in the name
python -m uv run pytest -x           # stop at the first failure
python -m uv run pytest -v           # print every test name
```

9.3 Worked example: change a Discovery dropdown and test it

1 Edit the YAML

Open `packages/domain/src/archscope_domain/config/discovery_options.yaml`. Find the question `id: communication` and add a line `- Microsoft Teams` under its `options:`, aligned with the other options (YAML is sensitive to spaces — use spaces, not tabs). Save with **Ctrl+S**.

2 Run the configuration tests

```
python -m uv run pytest packages/domain/tests/test_discovery_config.py
```

3 See it in the API

With the API running, call **GET /api/v1/discovery/question-bank** in Swagger: the new option is there (no restart needed).

4 See it in Streamlit

Refresh the Discovery Call Assistant page: the option appears in the dropdown.

TIP If the YAML has a mistake (wrong indentation, duplicate option, unknown input type), the Discovery page shows a list of exactly which entry is wrong instead of crashing. Fix it and refresh.

9.4 Golden tests: when an output change is intended

Golden tests fail whenever an output changes — even a deliberate improvement. That is the point: every change in output must be a *conscious decision*. When you intend a change:

1. Run the golden tests and read the diff to confirm the change is exactly what you meant.
2. Regenerate the approved files: `$env:GOLDEN_UPDATE="1"; python -m uv run pytest packages/domain/tests/golden; Remove-Item Env:GOLDEN_UPDATE`
3. Review the changed JSON files in VS Code's **Source Control** panel (left bar, branch icon) before committing — they are part of the code review.

9.5 The other quality checks (what CI runs)

Check	Command	What it catches
Lint	<code>python -m uv run ruff check packages apps</code>	Bugs-in-waiting and bad patterns (unused imports, risky code).
Format	<code>python -m uv run ruff format packages apps</code>	Inconsistent formatting. This command <i>fixes</i> it; CI uses <code>--check</code> .
Types	<code>python -m uv run mypy</code>	Type mistakes, e.g. passing text where a number is expected — before running anything.

Check	Command	What it catches
Architecture	<code>python -m uv run lint-imports</code>	A layer importing something it must not (Section 5.3).
Contract	<code>python -m uv run python -m archscope_api.openapi</code>	Regenerates <code>openapi.json</code> after API changes.
Tests	<code>python -m uv run pytest</code>	Wrong behaviour.

TIP Before every push, run the five commands in the table (or the cheat sheet in Section 15). If they are green on your computer, CI will almost always be green too.

10. Making changes safely (Git and CI)

10.1 The everyday Git workflow

1 Start a branch for your change

Never work directly on `main`. Give the branch a descriptive name.

```
git switch rebuild/platform
git pull
git switch -c feature/teams-dropdown
```

2 Make your change and run the checks

Edit files, then run the tests and checks from Section 9.5.

3 Review what changed

In VS Code open the **Source Control** panel (Ctrl+Shift+G). Click each file to see a side-by-side diff.
Or in the terminal:

```
git status
git diff
```

4 Commit (save a snapshot)

Write a message that says *why*, not just what.

```
git add -A
git commit -m "Discovery: add Microsoft Teams as a communication option"
```

5 Push to GitHub

```
git push -u origin feature/teams-dropdown
```

6 Open a Pull Request

Git prints a link after pushing; open it (or go to the repository on GitHub and click **Compare & pull request**). Describe the change, choose the base branch, and create the PR.

7 Wait for CI

On the PR page, the **Checks** section runs the workflow from `.github/workflows/ci-python.yml`. A green check = all passed. A red cross = click **Details** to read which step failed and why, fix it locally, commit and push again — the PR updates automatically.

8

Merge

When checks are green (and, in a team, someone reviewed it), click **Merge**.

10.2 What CI runs

Job	Steps
Lint · types · architecture	Install from <code>uv.lock</code> → ruff lint → ruff format check → mypy (strict) → import-linter contracts → OpenAPI contract is current.
Tests (Python 3.12 and 3.13)	Domain + golden parity → API integration tests → Streamlit smoke tests.
Domain on Python 3.11	Domain and integrations tests on 3.11, because the legacy Streamlit deployment may still use it.

10.3 Adding a new API endpoint (the recipe)

Follow the layers from the inside out:

1. **Domain:** if new business logic is needed, add a function to the right module in `packages/domain/src/archscope_domain/` with unit tests.
2. **Schema:** describe the request and response JSON as Pydantic classes in `apps/api/src/archscope_api/schemas/`.
3. **Service:** add a function in `services/` that calls the domain using domain types only.
4. **Router:** add the endpoint in `routers/v1/...py` with a `summary=`; map schema → domain → schema.
5. **Tests:** add API tests in `apps/api/tests/`.
6. **Contract:** run `python -m uv run python -m archscope_api.openapi`, then the full check list, then commit.

11. Configuration and secrets

Settings that differ between your computer, staging and production — or that are secret — are never written in the code. They come from **environment variables**, which locally are read from `.env`.

Variable	Used by	Purpose
<code>AUTH_DISABLED=1</code>	Streamlit	Skip sign-in on your computer only.
<code>ANTHROPIC_API_KEY</code>	Streamlit, API	Key for AI drafting (Discovery). Get it at console.anthropic.com → API Keys.
<code>ANTHROPIC_MODEL</code>	API	Optional: override the AI model.
<code>HUBSPOT_TOKEN</code>	Streamlit	Optional: a default HubSpot private app token.
<code>ARCHSCOPE_ENVIRONMENT</code>	API	<code>local</code> , <code>test</code> , <code>staging</code> or <code>production</code> . Production hides the docs page.
<code>ARCHSCOPE_CORS_ORIGINS</code>	API	Websites allowed to call the API, as a JSON list, e.g. <code>["https://app.example.com"]</code> .
<code>ARCHSCOPE_LOG_LEVEL</code>	API	<code>INFO</code> normally, <code>DEBUG</code> when investigating.

WATCH OUT **Secrets never go into Git.** Not in code, not in comments, not in test files. If a key is committed by mistake, deleting the file is not enough — it stays in history: revoke the key in its provider's dashboard and create a new one.

In production, secrets are stored in the hosting platform's secret manager (Section 12.5), not in files.

12. How it will be deployed

GOOD TO KNOW This section describes the **plan** for Phases 2–4. The Dockerfiles and hosting are not built yet; the decisions below are the intended design and may be refined.

12.1 Today

The legacy app is deployed on **Streamlit Community Cloud**, which installs `requirements.txt` and runs `Home.py`. `requirements.txt` now also installs the local `packages/domain` and `packages/integrations`. Before merging the rebuild branch into the deployed branch, do a test deploy from `rebuild/platform` to confirm Streamlit Cloud installs them.

12.2 The target: containers

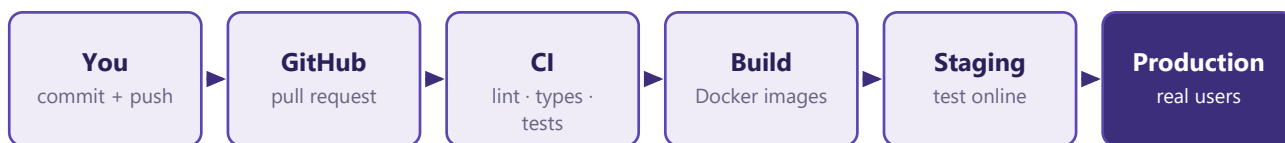
Each service is packaged as a **Docker image** — a sealed box containing the code, the exact libraries and the operating-system bits it needs. The same image runs on your laptop, in CI, on staging and in production, which removes "works on my machine" problems.

- **Multi-stage Dockerfiles:** a first stage installs and builds; the final, small stage only contains what is needed to run (smaller, faster, fewer security holes). One for `apps/api`, one for `apps/web`.
- **Docker Compose** for local development: one command (`docker compose up`) starts the API, the web app, PostgreSQL, Redis and object storage together.
- The API runs as a **non-root user** and exposes `/healthz` and `/readyz` so the platform knows when it is alive and ready to receive traffic.

12.3 Environments

Environment	Who uses it	Data	Deployed when
Local	You	Your own test data	When you run it
Staging	The team, for final checks	Fake/sanitized data	Automatically, on every merge to the main branch
Production	Real users	Real client data	Manually approved promotion of a build already tested in staging

12.4 The delivery pipeline



Each arrow only happens if the previous step is green. A red CI run blocks the merge.

Figure 4 — From your commit to production.

1. You push a branch and open a Pull Request; CI runs every check.
2. On merge, CI builds the Docker images, tags them with the commit ID, and pushes them to a container registry.
3. Database migrations (Alembic) run automatically against staging, then the new images are deployed there.
4. After checking staging, someone approves promotion: the *same* images go to production (never rebuilt — what was tested is what ships).
5. If something goes wrong, **roll back** by redeploying the previous image tag.

12.5 Hosting options

Option	Good for	Notes
Render / Railway / Fly.io	Small team, first production launch	Simplest: deploy containers + managed Postgres/Redis with a few clicks; built-in HTTPS and secrets.
Vercel (web) + a container host (API)	Best Next.js experience	Vercel is made by the Next.js team; the API stays on a container host.
Azure Container Apps / AWS ECS / Google Cloud Run	Company standards, larger scale	More setup, more control; managed databases, secret managers, private networks.

A sensible first choice is a managed platform (Render or similar) for all services, moving to a large cloud only when scale or company policy requires it. The architecture doesn't change between them — that is a benefit of containers.

12.6 Database changes: migrations

In Phase 2 the database structure is versioned with **Alembic**. Every change to tables is a small *migration* script stored in Git, applied automatically on deploy, in order. Rule of thumb: migrations must keep the previous app version working (add columns first, remove old ones in a later release), so deploys and rollbacks are safe.

12.7 Production checklist (before the first real launch)

- HTTPS on a proper domain; `ARCHSCOPE_CORS_ORIGINS` set to that domain only.
- Google sign-in with domain allow-list and roles; `AUTH_DISABLED` absent.
- Secrets in the platform's secret manager; HubSpot tokens encrypted at rest (Phase 2).
- Automatic daily database backups — and one *tested* restore.
- Error tracking and log search connected; alerts to a channel someone watches.
- Docs page disabled (`ARCHSCOPE_ENVIRONMENT=production`).

13. Scaling and escalation

Scaling ("escalar") means handling more users, more clients and more data without slowing down. The architecture was chosen so scaling is a series of small, independent steps rather than a rewrite.

13.1 Why it can scale

- **The API is stateless:** it keeps nothing in memory between requests (state lives in PostgreSQL/Redis). So you can run 1, 3 or 10 copies behind a load balancer and any copy can answer any request.
- **Slow work moves to workers:** big portal pulls and document rendering go into a queue (Redis) processed by separate worker processes, so the website stays fast.
- **The domain is pure:** no shared state, safe to run in parallel.
- **Tenancy-ready:** every table carries `org_id`, so becoming a multi-company SaaS is a configuration change, not a rewrite.

13.2 The scaling path, step by step

Stage	Signal that you need it	What to do
1. One instance	Launch; a single team uses it.	1 API container, 1 worker, managed Postgres + Redis.
2. Scale out the API	Response times rise at busy hours; CPU often above ~70%.	Run 2–4 API copies behind the load balancer (a setting on most platforms). Enable autoscaling on CPU.
3. More workers	Portal pulls / PDF jobs wait in the queue.	Add worker copies; split queues (fast vs. slow jobs).
4. Cache	The same expensive results are computed again and again.	Cache portal snapshots and question banks in Redis with sensible expiry.
5. Database tuning	Slow queries appear in monitoring.	Add indexes for the slow queries, connection pooling (PgBouncer), then read replicas for reports.
6. External limits	HubSpot 429 (rate limit) errors or high AI bills.	Per-portal request budgets and backoff (already partly built), job throttling; AI usage limits per organization.
7. Multi-tenant SaaS	Other companies want to use it.	Enable sign-up per organization, per-org limits and billing; data already separated by <code>org_id</code> .

TIP Scale on **evidence**, not guesses. Each step should be triggered by a number from monitoring (response time, queue length, error rate), not by intuition.

13.3 Observability: seeing what production is doing

- **Logs:** every request is logged with its `X-Request-ID`. When a user reports an error, ask for the `trace_id` shown in the error and search the logs for it.
- **Metrics:** requests per second, response time, error rate, queue length, database load — on a dashboard.
- **Tracing** (OpenTelemetry, planned): follow one request through API → worker → database → HubSpot to see where time is spent.
- **Alerts:** notify a person when error rate or response time crosses a threshold.

13.4 Escalating incidents (when something breaks)

Level	Example	Response
Sev 1 — Critical	Production down, data exposed, or anything writing to a client portal.	Act immediately: roll back to the previous image, inform stakeholders, then investigate. Write a short post-mortem afterwards.
Sev 2 — Major	One module broken for everyone (e.g. PDF export fails).	Fix or roll back the same day; tell users of a workaround.
Sev 3 — Minor	Cosmetic bug, a rare edge case.	Create a ticket, fix in a normal release.

For every incident: (1) note the time and the `trace_id`, (2) check `/healthz` and the logs, (3) decide roll back vs. fix forward, (4) add a test that reproduces the bug before fixing it, so it can never come back.

14. Troubleshooting

Symptom	Likely cause	Fix
<code>python -m uv</code> → "No module named uv"	uv not installed for this Python.	<code>python -m pip install --user uv</code>
'uv' is not recognized...	You typed <code>uv</code> instead of <code>python -m uv</code> .	Use <code>python -m uv ...</code> as in this guide.
<code>ModuleNotFoundError: archscope_api</code> (or <code>fastapi</code>)	Libraries not installed, or you ran plain <code>python</code> instead of <code>python -m uv run</code> .	<code>python -m uv sync --all-packages</code> , then always prefix with <code>python -m uv run</code> .
"address already in use" / port 8000 or 8501 busy	The server is already running in another terminal (or a stuck one).	Use the other terminal, or stop it with Ctrl+C. Or pick another port: <code>--port 8001</code> .
Browser: "This site can't be reached"	The server isn't running or crashed.	Look at its terminal for a red error; start it again (Section 7).
Streamlit shows a sign-in page	<code>AUTH_DISABLED=1</code> missing from <code>.env</code> .	Add it (Section 6, step 7) and restart Streamlit.
API: <code>400 hubspot-token-required</code>	No token sent.	Swagger → Authorize (Section 8.8).
API: <code>502 hubspot-access-denied</code>	Token invalid/expired or missing scopes.	Create a new private app token with the README's read scopes.
API: <code>503 ai-not-configured</code>	No <code>ANTHROPIC_API_KEY</code> .	Add it to <code>.env</code> , restart the API.
API: <code>422 validation-error</code>	Body doesn't match the schema.	Read <code>errors[].loc</code> — it names the field. Compare with the Schemas section in Swagger.
A golden test fails after your change	Your change altered an output.	If unintended: fix the code. If intended: Section 9.4.
<code>test_openapi_is_committed_and_current</code> fails	API changed, contract not regenerated.	<code>python -m uv run python -m archscope_api.openapi</code> and commit <code>openapi.json</code> .
lint-imports: a contract is BROKEN	A layer imported something forbidden.	Move the code to the right layer (Section 5.3).
Discovery page lists YAML errors	Mistake in <code>discovery_options.yaml</code> .	Fix the entries it names (indentation, duplicates, input type).

15. Cheat sheet

Setup (once)

```
python -m pip install --user uv
git switch rebuild/platform
python -m uv sync --all-packages
```

Run

```
python -m uv run streamlit run Home.py # http://localhost:8501
python -m uv run uvicorn archscope_api.main:app --reload # http://localhost:8000/docs
# stop either one: Ctrl+C in its terminal
```

Check everything before you push

```
python -m uv run ruff format packages apps
python -m uv run ruff check packages apps
python -m uv run mypy
python -m uv run lint-imports
python -m uv run python -m archscope_api.openapi
python -m uv run pytest -q
```

Git

```
git status # where am I, what changed?
git switch -c feature/my-change # new branch
git add -A
git commit -m "Explain why"
git push -u origin feature/my-change
```

Useful addresses

Address	What
http://localhost:8501	Streamlit app
http://localhost:8000/docs	API interactive docs (Swagger)
http://localhost:8000/healthz	API alive?
http://localhost:8000/readyz	API ready (configuration loads)?
http://localhost:8000/openapi.json	The API contract, machine-readable

Appendix A — Endpoint reference

All business endpoints start with `/api/v1`. Every one is documented in Swagger with its full request and response schema.

Group	Method	Path (after /api/v1, except health)	Purpose
health	GET	<code>/healthz</code>	Liveness: the process is running.
health	GET	<code>/readyz</code>	Readiness: rules and question bank load.
architecture	POST	<code>/blueprints</code>	Generate an architecture blueprint.
architecture	POST	<code>/blueprints/markdown</code>	Render a blueprint as Markdown.
architecture	POST	<code>/diagrams/from-blueprint</code>	Diagram (nodes/edges) of a proposed architecture.
architecture	POST	<code>/diagrams/from-snapshot</code>	Diagram of a live portal, coloured by audit risk.
architecture	POST	<code>/diagrams/render?format=png dot drawio</code>	Render a diagram to a file.
portal	POST	<code>/portal/snapshot</code>	Pull a read-only portal snapshot (token).
portal	POST	<code>/portal/snapshot/document?format=md docx</code>	Portal documentation.
portal	POST	<code>/audits</code>	Best-practice findings + counts (token).
portal	POST	<code>/property-audits</code>	Keep / Review / Remove rating per property (token).
portal	POST	<code>/property-audits/export?format=xlsx pdf</code>	Property audit workbook or PDF.
portal	POST	<code>/reports/context</code>	Executive report figures.
portal	POST	<code>/reports/document?audience=client internal</code>	Executive report .docx.
proposals	GET	<code>/proposals/default</code>	New proposal with standard boilerplate.
proposals	GET	<code>/proposals/libraries</code>	Reusable scope items and add-ons.
proposals	POST	<code>/proposals/analysis</code>	Pricing, coverage, scope groups, warnings.
proposals	POST	<code>/proposals/import/wrd</code>	Merge a Requirements Document.
proposals	POST	<code>/proposals/import/jep</code>	Merge a Joint Evaluation Plan.
proposals	POST	<code>/proposals/scope/library-items</code>	Add scope-library items.
proposals	POST	<code>/proposals/scope/library-addons</code>	Add add-on services.
proposals	POST	<code>/proposals/scope/from-requirements</code>	Draft scope for uncovered requirements.
proposals	POST	<code>/proposals/template</code>	Strip deal content, keep boilerplate.
proposals	POST	<code>/proposals/pdf</code>	Branded proposal / SOW PDF.

Group	Method	Path (after /api/v1, except health)	Purpose
discovery	GET	/discovery/question-bank	Configured questions and options.
discovery	POST	/discovery/business-analysis/payload	Preview the exact AI payload.
discovery	POST	/discovery/business-analysis	AI business analysis (SSE stream).
discovery	POST	/discovery/client-explainer	AI client explainer (SSE stream).
discovery	POST	/discovery/suggest-modules	Hubs recommended by an analysis.
exports	POST	/exports/markdown?format=docx pdf	Markdown to Word or PDF.
exports	POST	/exports/sections?format=docx xlsx csv pdf	Table sections to a document.

Appendix B — Key files

File	Why you would open it
<code>packages/domain/src/archscope_domain/config/rules.yaml</code>	Change best-practice rules.
<code>packages/domain/src/archscope_domain/config/discovery_options.yaml</code>	Change Discovery questions and dropdowns.
<code>packages/domain/src/archscope_domain/ports.py</code>	See the read-only interface to HubSpot.
<code>apps/api/src/archscope_api/main.py</code>	How the API is assembled (middleware, routers).
<code>apps/api/src/archscope_api/routers/v1/</code>	Every endpoint's HTTP code.
<code>apps/api/src/archscope_api/schemas/</code>	Every request/response JSON shape.
<code>apps/api/src/archscope_api/settings.py</code>	All API settings and environment variables.
<code>apps/api/openapi.json</code>	The generated API contract.
<code>pyproject.toml</code>	Workspace members, tool settings, architecture contracts.
<code>.github/workflows/ci-python.yml</code>	What CI runs.
<code>README.md</code>	Module-by-module reference for the Streamlit app and platform overview.